TU BIBLIOTECA · ORDENADA PARA BACKEND JAVA → ARQUITECTURA FINTECH

Ruta de lectura

Los 21 libros de tu carpeta en el orden en que conviene leerlos, pensado para alguien que está aprendiendo Spring Boot desde cero, busca su primer trabajo y apunta a arquitecto en el sector financiero. La línea principal tiene 5 fases; IA y bajo nivel van en ramales paralelos para que no frenen la línea principal.

Spring Boot · Oficio · Primer trabajo · Arquitectura · Distribuidos · Ramal IA · Ramal bajo nivel

Un libro principal a la vez

Uno que leés de corrido y, como mucho, uno de consulta abierto al lado. Las fases 2 y 3 sí van en paralelo entre sí.

Cada capítulo, a tu código

Cada estación trae un ejercicio concreto sobre Billetera Clara o apifintech. Si no lo podés hacer, el capítulo todavía no quedó.

Notas a mano, con un caso

En Obsidian, por capítulo: la idea en una frase y un ejemplo concreto tuyo. Sin ejemplo propio, la nota no sirve para repasar.

1

Spring Boot de verdad

Ahora, en paralelo con la diplomatura. Objetivo: entender qué hace Spring por debajo de cada anotación que ya usás.



ESTACIÓN 1

☐ Leído

Spring Start Here

Laurențiu Spilcă · Manning · ~400 págs

Está escrito para quien arranca Spring desde cero. Explica el contexto de Spring, los beans y la inyección de dependencias antes de llegar a REST, acceso a datos, transacciones y tests. Es exactamente lo que Spring Boot te esconde.

> **Llevalo a tu código:** Por cada capítulo, buscá en apifintech dónde aparece lo que leíste (@Service, @Transactional, @RestController) y anotá qué pasaría si sacaras esa anotación. Después probalo.

Edición: Tu PDF es la 1ª edición (Spring 5). La 2ª edición, con Spring 7 y Spring Boot 4, está en acceso anticipado en Manning con publicación estimada para diciembre de 2026. Los conceptos de la 1ª siguen valiendo.



ESTACIÓN 2

☐ Leído

Spring in Action

Craig Walls · Manning · ~520 págs

Segunda pasada, más amplia: Spring Data, Spring Security, mensajería, programación reactiva y Actuator. No hace falta leerlo de corrido; usalo por tema cuando lo necesites.

> **Llevalo a tu código:** Antes de agregar login a Billetera Clara, leé los capítulos de Spring Security y protegé un solo endpoint. Recién después extendelo al resto.

Edición: La 6ª edición (2022) es la más reciente. Es anterior a Spring Boot 3, así que vas a ver paquetes javax.* donde hoy se usa jakarta.*.

Oficio: código que otro pueda leer

Cuando ya armás un CRUD sin guía. Es la base para leer repos grandes como Kill Bill o Fineract.



ESTACIÓN 3

Leído

A Philosophy of Software Design

John Ousterhout · ~190 págs

El más corto y el que más cambia cómo pensás el diseño. Su idea central: el enemigo es la complejidad, y la combatís con módulos profundos, que esconden mucho detrás de una interfaz chica.

> **Llevalo a tu código:** Tomá el servicio de cuentas de apifintech: contá sus métodos públicos y cuánto hace cada uno. Si exponés cinco métodos para lo que el controlador ve como una sola operación, juntalos.

Edición: Conviene la 2ª edición (2021), que revisa su postura sobre comentarios y métodos cortos.



ESTACIÓN 4

Leído

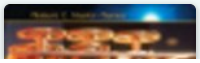
Refactoring

Martin Fowler · ~450 págs

Un catálogo de cambios chicos y seguros para mejorar código sin romperlo, siempre con tests verdes entre paso y paso. De paso, te obliga a escribir tests, uno de tus puntos flojos.

> **Llevalo a tu código:** Aplicá tres refactorings en Billetera Clara: Extract Function, Introduce Parameter Object y Replace Conditional with Polymorphism. Corré los tests después de cada paso.

Edición: La 2ª edición (2018) tiene los ejemplos en JavaScript; la 1ª (1999) los tenía en Java. Las técnicas se pasan a Java sin problema.



ESTACIÓN 5

Leído



Working Effectively with Legacy Code

Michael Feathers · ~450 págs

Es justo la habilidad que querés desarrollar: modificar código que no escribiste sin romperlo. Te enseña a encontrar "seams" (puntos donde podés cortar dependencias) y a escribir tests de caracterización.

> **Llevalo a tu código:** En el próximo repo de tu ruta (Joda-Money, Kill Bill), antes de tocar una clase escribí un test que fije lo que hace hoy, aunque parezca raro. Recién entonces cambiala.

Edición: Ejemplos en Java, C++ y C. Es de 2004 pero no envejeció.



ESTACIÓN 6 · CONSULTA

☐ Leído

Code Complete

Steve McConnell · 2ª ed. · ~900 págs

Una enciclopedia de la construcción de software. No lo leas de tapa a tapa: tenelo como consulta para nombres de variables, rutinas, programación defensiva y depuración.

> **Llevalo a tu código:** Usá sus checklists al final de cada capítulo como revisión antes de hacer un commit grande o pedir code review.

Edición: La 2ª edición (2004) es la vigente. Los ejemplos mezclan varios lenguajes, incluido Java.

3 Primer trabajo

En paralelo con la fase 2: un poco cada día. Junto con tus PDFs de preguntas y la prueba técnica.



ESTACIÓN 7

☐ Leído



Cracking the Coding Interview

Gayle Laakmann McDowell · ~700 págs

Estructuras de datos, algoritmos y cómo encarar una entrevista técnica. Leé las explicaciones de cada tema y resolvé los problemas en Java.

> **Llevalo a tu código:** Un problema por día, cronometrado, en Java y sin IA. Anotá en Obsidian solo el patrón que lo resolvía (dos punteros, hash map, BFS).

Edición: En enero de 2025 salió Beyond Cracking the Coding Interview (McDowell, Mroczka, Lerner y Mamano), más actual. Si lo conseguís, usá ese como principal.



ESTACIÓN 8

☐ Leído

Programming Pearls

Jon Bentley · 2ª ed. · ~250 págs

Columnas cortas sobre pensar el problema antes de programar: estimaciones rápidas, búsqueda binaria correcta, rendimiento. Es un buen puente hacia system design.

> **Llevalo a tu código:** Hacé la estimación al estilo del libro: si Billetera Clara tuviera 100.000 usuarios con 5 movimientos diarios, ¿cuántas escrituras por segundo en el pico? ¿Cuánto disco por año?

4 De programador a arquitecto

Cuando ya tenés un proyecto propio con varias capas. Acá aparece el dominio financiero de la mano de DDD.

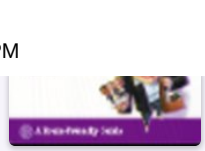


ESTACIÓN 9

☐ Leído

Head First Software Architecture

Gandhi, Richards y Ford · 2024 · ~500 págs



La entrada más amable a la arquitectura: características, decisiones y estilos (en capas, monolito modular, microkernel, microservicios, orientado a eventos), con muchos dibujos y ejercicios.

> **Llevalo a tu código:** Elegí qué estilo tiene hoy tu proyecto y escribí en una página por qué, con las características que más importan en una billetera (consistencia, seguridad, auditabilidad).



ESTACIÓN 10

☐ Leído

Fundamentals of Software Architecture

Mark Richards y Neal Ford · ~420 págs

El mismo terreno que Head First, más profundo y formal. Agrega cómo medir la modularidad, cómo documentar decisiones y qué hace un arquitecto en el día a día.

> **Llevalo a tu código:** Escribí tu primer ADR (registro de decisión de arquitectura) para una decisión real de Billetera Clara, por ejemplo base relacional o no.

Edición: Hay 2ª edición de marzo de 2025 con cinco capítulos nuevos, incluidos IA generativa, nube y topologías de datos y equipos.



ESTACIÓN 11

☐ Leído

Clean Architecture

Robert C. Martin · ~430 págs

La regla de dependencias, los puertos y adaptadores, y SOLID aplicado a componentes. Leelo con espíritu crítico: Ousterhout discute varias ideas del mismo autor, y esa tensión te enseña a decidir.

> **Llevalo a tu código:** Dibujá las dependencias entre paquetes de apifintech. ¿Tu dominio importa algo de Spring o de JPA? Si sí, probá separar una entidad de dominio de su entidad JPA.



ESTACIÓN 12 · CONSULTA

Leído

Patterns of Enterprise Application Architecture

Martin Fowler · ~530 págs

Los patrones que están dentro de JPA y Hibernate, con nombre propio: Unit of Work, Identity Map, Data Mapper, Lazy Load, Optimistic Offline Lock.

> **Llevalo a tu código:** Por cada patrón, encontralo en Hibernate: el contexto de persistencia es Identity Map más Unit of Work, y @Version es Optimistic Offline Lock. Anotá cada par en Obsidian.

Edición: Es de 2002, pero sigue siendo la referencia para entender cualquier ORM.



ESTACIÓN 13

Leído

Domain-Driven Design

Eric Evans · ~560 págs

Cómo modelar un dominio complejo junto con los expertos del negocio: lenguaje ubicuo, entidades, agregados y contextos delimitados. Es denso; leé las partes I y II primero y después la IV (diseño estratégico).

> **Llevalo a tu código:** Armá el glosario de tu billetera con tu Manual de dominio FinTech al lado: Cuenta, Movimiento, Saldo, Transferencia. Definí cuál es el agregado y qué regla protege.

5 Sistemas distribuidos y producción

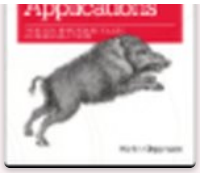
El corazón de la arquitectura FinTech: datos que no se pueden perder, servicios que fallan y dinero que no se puede duplicar.



ESTACIÓN 14

Leído

Designing Data-Intensive Applications



Martin Kleppmann · ~600 págs

El libro central para FinTech: replicación, particionado, transacciones y niveles de aislamiento, consistencia y procesamiento de streams. Explica por qué un saldo se puede corromper.

> **Llevalo a tu código:** Reproducí un lost update en PostgreSQL con dos sesiones que descuentan del mismo saldo. Después arreglalo con SELECT ... FOR UPDATE y, aparte, con @Version.

Edición: En marzo de 2026 salió la 2ª edición, con Chris Riccomini como coautor. Si tu PDF es la 1ª (2017), sigue valiendo.



ESTACIÓN 15

☐ Leído

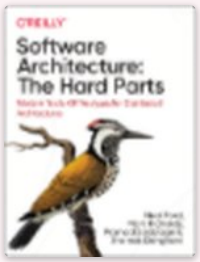
Building Microservices

Sam Newman · ~600 págs

Cuándo conviene partir un sistema, cómo hacerlo y qué cuesta: comunicación entre servicios, sagas, despliegue, testing y observabilidad.

> **Llevalo a tu código:** Partí Billetera Clara en dos servicios (cuentas y notificaciones) que se hablen por un evento, y anotá todo lo que se volvió más difícil.

Edición: La 2ª edición (2021) reescribe buena parte del libro. Es la que conviene.



ESTACIÓN 16

☐ Leído

Software Architecture: The Hard Parts

Ford, Richards, Sadalage y Dehghani · ~460 págs

Los dilemas de los sistemas distribuidos: cómo separar datos, transacciones distribuidas, sagas y contratos. Todo con un caso de estudio que sigue a un equipo durante el libro.

> **Llevalo a tu código:** Para una transferencia entre dos servicios, compará dos tipos de saga del libro y elegí una con su análisis de pros y contras.



ESTACIÓN 17

Leído

Release It!

Michael Nygard · ~350 págs

Cómo falla el software en producción y cómo evitar que una falla tumbe todo: timeouts, circuit breaker, bulkheads.

> **Llevalo a tu código:** Simulá un proveedor de pagos lento y protegé la llamada con timeout y circuit breaker usando Resilience4j. Mirá qué pasa con y sin protección.

Edición: Conviene la 2ª edición (2018).



ESTACIÓN 18

Leído

System Design Interview

Alex Xu · vol. 1 · ~320 págs

Práctica de diseño de sistemas: rate limiter, generador de IDs únicos, sistema de notificaciones. Sirve para entrevistas de nivel medio y para aplicar lo de DDIA.

> **Llevalo a tu código:** Hacé cada capítulo primero vos en papel, 45 minutos, y recién después leé la solución del autor.



ESTACIÓN 19 · CONSULTA

Leído

Software Engineering at Google

Winters, Manshreck y Wright · ~600 págs

Cómo trabaja un equipo grande con código que dura años: code review, testing a escala, dependencias, documentación. Leé selectivamente, empezando por los capítulos de testing.

> **Llevalo a tu código:** Escribí una guía de estilo y de review de una página para tu propio proyecto, como si entrara un compañero

mañana.

Ramal IA

IA

Va por tu propio plan de IA, que arranca por matemáticas. Este libro es la parte de producto.



RAMAL IA

Leído

AI Engineering

Chip Huyen · 2025 · ~530 págs

Cómo construir aplicaciones sobre modelos fundacionales: evaluación, prompts, RAG, agentes, fine-tuning y optimización de inferencia. Es la parte de producto de tu plan de IA.

> **Llevalo a tu código:** Armá un RAG mínimo sobre tus notas de Obsidian y medí si responde bien con 20 preguntas que escribas vos.

Ramal bajo nivel

BN

Para tus ratos libres: sistemas operativos, memoria y cómo corre realmente un programa.



RAMAL BAJO NIVEL

Leído

Programming from the Ground Up

Jonathan Bartlett · ~330 págs · gratis

Programar en assembly x86 sobre Linux: registros, memoria, la pila, llamadas al sistema y cómo se llama una función. Después de esto, la JVM deja de ser magia.

> **Llevalo a tu código:** Hacé los ejemplos en tu Arch. Son de 32 bits, así que ensamblá con as --32 y enlazá con

```
ld -m elf_i386.
```

Edición: Es libre y se distribuye gratis.

TUS PDFS DE PRÁCTICA

Dónde encaja tu material propio

No son libros publicados, así que no los busqué en internet. Te marco en qué fase usar cada uno.



500 ejercicios de lógica

Calentamiento diario de 20 min.

FASE 1 → 3



Guía de reconstrucción Spring Boot + Maven

Tu método de reconstruir sin IA.

FASE 1



Grandes proyectos personales Java + Spring Boot

De acá sale el portfolio.

FASE 1 → 2



600 preguntas Java backend

Con tu carpeta de respuestas.

FASE 3



Prueba técnica Spring Boot desde cero

Hacela cronometrada, como una real.

FASE 3



Manual de dominio FinTech

Leelo junto con DDD.

FASE 4



1000 preguntas sistema bancario / startup

Repaso del dominio y del diseño.

FASE 4 → 5



Simulación proyecto final diplomatura FinTech

Usalo para aplicar DDIA y microservicios.

FASE 5

Fuera de la ruta: **claudecode-cheatsheet** es una herramienta, no lectura. **Quicksand_curso_completo** no lo pude identificar por el título; la miniatura parece un curso de diseño o tipografía, así que lo dejé afuera.

PARA SUMAR A LA CARPETA

Libros que te faltan

Elegidos por los huecos de tu biblioteca: tus puntos flojos (excepciones, streams, testing), la persistencia que usás todos los días y tus intereses de bajo nivel.

Java base · antes o durante la fase 1

Effective Java, 3ª ed.

Joshua Bloch

90 reglas cortas sobre Java bien escrito, incluidas excepciones, genéricos y lambdas. Es el libro que más te falta.

Modern Java in Action

Urma, Fusco y Mycroft

Streams, lambdas y Optional explicados con ejemplos. Ataca directo uno de tus puntos flojos.

Java Concurrency in Practice

Brian Goetz y otros

Hilos, locks y estado compartido. En FinTech dos transferencias al mismo tiempo son el caso normal.

Testing y persistencia · fase 2

Unit Testing Principles, Practices, and Patterns

Vladimir Khorikov

Qué testear, qué no y cuándo usar mocks. Ninguno de tus libros cubre testing de forma central.

High-Performance Java Persistence

Vlad Mihalcea

JPA, Hibernate y transacciones en serio: N+1, locking, niveles de aislamiento. Lo usás todos los días.

API Security in Action

Neil Madden

Autenticación, tokens y ataques a APIs, con ejemplos en Java. En finanzas la seguridad no es opcional.

Distribuidos y FinTech · fases 4 y 5

Microservices Patterns

Chris Richardson

Sagas, eventos y consistencia con ejemplos en

Bajo nivel e IA · ramales

Computer Systems: A Programmer's Perspective

Sagas, eventos y consistencia con ejemplos en Java y Spring. Complementa a Newman con código.

System Design Interview, Vol. 2

Alex Xu y Sahn Lam

Trae diseños de un sistema de pagos y de una billetera digital, justo tu proyecto.

Kafka: The Definitive Guide

Shapira, Palino, Sivaram y Petty

Kafka está en tu ruta de repositorios; este libro te deja leer su código con contexto.

Bryant y O'Hallaron

El siguiente paso natural después de Ground Up: memoria, caché, enlazado, procesos.

Operating Systems: Three Easy Pieces

Remzi y Andrea Arpaci-Dusseau

Sistemas operativos claros y gratis online. Lo podés practicar en tu Arch.

Crafting Interpreters

Robert Nystrom

Construís un lenguaje desde cero; el primer intérprete está escrito en Java. Gratis online.

Mathematics for Machine Learning

Deisenroth, Faisal y Ong

Encaja con tu decisión de empezar IA por matemáticas. Gratis en PDF.

Build a Large Language Model (From Scratch)

Sebastian Raschka

Para después de AI Engineering: construir un LLM pequeño entendiendo cada pieza.

Páginas aproximadas. El progreso se guarda solo en este navegador. Fuentes consultadas para ediciones vigentes:

- Kleppmann: DDIA 2ª edición
- O'Reilly: Fundamentals of Software Architecture, 2ª ed.
- Manning: Spring Start Here, 2ª ed.
- Manning: Spring in Action, 6ª ed.
- Beyond Cracking the Coding Interview
- O'Reilly: Head First Software Architecture
- O'Reilly: Building Microservices, 2ª ed.
- O'Reilly: Release It!, 2ª ed.
- O'Reilly: Software Architecture: The Hard Parts
- Programming from the Ground Up
- AI Engineering, Chip Huyen